

Lab 4: Limited Dependent Variable Models

SOCIOL 723, Social Statistics II, Thursday, September 17

Wenhao Jiang

Table of contents

1	Goals	2
2	Binary outcomes	2
2.1	Three scales, one coefficient	3
2.2	Start with predicted probabilities	3
2.3	Then marginal effects: one number per variable	5
2.4	Odds ratios, at most as a footnote	6
2.5	Uncertainty: robust and clustered standard errors	6
2.6	Probit: another S-curve	7
3	Interactions	8
4	Outcomes with several categories	10
4.1	Multinomial logit: a nominal outcome	10
4.2	Ordered logit: an ordered outcome	11
5	Count outcomes	13
5.1	Is it overdispersed?	14
5.2	Excess zeros, and the hurdle model	15
5.3	Comparing the count models by AIC and BIC	17
5.4	Does the model reproduce the data?	18
6	Multilevel models	19
6.1	Reading the <code>lmer</code> formula	19
6.2	Predictors at both levels	20
6.3	The two variances and the intraclass correlation	20
6.4	A random-intercept logit	21
7	Exercises	23
8	Session information	24

1 Goals

1. Fit a logit, then read it on the probability scale: predicted probabilities first, average marginal effects second, odds ratios as a footnote.
2. Handle an interaction in a nonlinear model: when to include the product, and how to report the interaction.
3. Fit a multinomial logit on a nominal outcome and an ordered logit on an ordered one.
4. Fit count models: Poisson, the three overdispersion repairs, the hurdle model; compare them by AIC and BIC.
5. Fit a random-intercept model, read the two variances and the intraclass correlation, and compare it with fixed effects.

```
library(lme4)      # lmer, glmer
library(dplyr)
library(ggplot2)
library(marginaleffects)
library(sandwich); library(lmtest)
library(MASS)      # glm.nb, polr
library(AER)       # dispersiontest
library(nnet)      # multinom
library(pscl)      # hurdle, zeroinfl

gss <- readRDS("../Data/gss_earnings.rds")
```

2 Binary outcomes

Outcome: `ba`, whether the respondent holds a bachelor's degree or more. The model is Tuesday's: parents' education, vocabulary score, female, black.

```
f <- ba ~ pareduc + wordsum + female + black

m_logit <- glm(f, data = gss, family = binomial(link = "logit"))
m_probit <- glm(f, data = gss, family = binomial(link = "probit"))
m_lpm <- lm(f, data = gss)

## the logit table from the slides: coefficient, SE, z, exp(coefficient)
round(cbind(coef(summary(m_logit))[, 1:3], exp_b = exp(coef(m_logit))), 3)
#>               Estimate Std. Error z value exp_b
#> (Intercept)   -5.810      0.245 -23.678 0.003
#> pareduc        0.207      0.013  15.389 1.229
#> wordsum        0.391      0.024  16.281 1.479
#> female         0.279      0.078   3.592 1.321
#> black        -0.114      0.110  -1.042 0.892
```

2.1 Three scales, one coefficient

The `wordsum` coefficient is 0.391 on the log-odds scale, $e^{0.391} = 1.48$ on the odds scale, and *not one number* on the probability scale. The first two are exact and uninterpretable to most readers; the third is what the reader wants and depends on where the person starts.

```
## one person at a time: log-odds, odds, probability for wordsum = 3, 6, 9
## with the other covariates at pareduc = 12, female = 0, black = 0
b <- coef(m_logit)
xb <- b["(Intercept)"] + b["pareduc"] * 12 + b["wordsum"] * c(3, 6, 9)
round(cbind(wordsum = c(3, 6, 9), log_odds = xb, odds = exp(xb),
            prob = plogis(xb)), 3)
#>      wordsum log_odds odds prob
#> [1,]      3   -2.158 0.116 0.104
#> [2,]      6   -0.983 0.374 0.272
#> [3,]      9    0.191 1.210 0.548
```

Each step of three words adds the same $3 \times 0.391 = 1.17$ to the log-odds and multiplies the odds by the same $e^{1.17} = 3.23$; the probability moves by a different amount each time.

2.2 Start with predicted probabilities

How likely is a degree if your parents finished high school (12 years) rather than college (16)? The observed-value approach: set `pareduc` to 12 for *everyone*, keep every other covariate at its actual value, average the fitted probabilities; repeat at 16.

```
## avg_predictions() does exactly that loop, with a delta-method interval
avg_predictions(m_logit, variables = list(pareduc = c(8, 12, 16, 20)))
#>
#> pareduc Estimate Std. Error    z Pr(>|z|)      S 2.5 % 97.5 %
#>      8      0.198     0.01279 15.5 <0.001  177.2 0.173 0.223
#>     12      0.343     0.00912 37.6 <0.001 1027.2 0.325 0.361
#>     16      0.521     0.01098 47.5 <0.001    Inf 0.500 0.543
#>     20      0.694     0.01795 38.7 <0.001    Inf 0.659 0.729
#>
#> Type: response
```

```
## the contrast we report: high school to college, 18 points
avg_comparisons(m_logit, variables = list(pareduc = c(12, 16)))
#>
#> Estimate Std. Error    z Pr(>|z|)      S 2.5 % 97.5 %
#>    0.178     0.0117 15.2 <0.001 170.4 0.155 0.201
#>
#> Term: pareduc
#> Type: response
```

```
#> Comparison: 16 - 12
```

```
## the counterfactual grid sets pareduc to each value for EVERYONE, then averages;  
## plain `by = "pareduc"` would instead average fitted values among the people  
## who actually have that pareduc, which mixes in their other covariates  
plot_predictions(m_logit, by = "pareduc",  
                 newdata = datagrid(pareduc = 8:20, grid_type = "counterfactual")) +  
  labs(x = "Parents' education (years)", y = "Pr(bachelor's or more)") +  
  theme_minimal(base_size = 10)
```

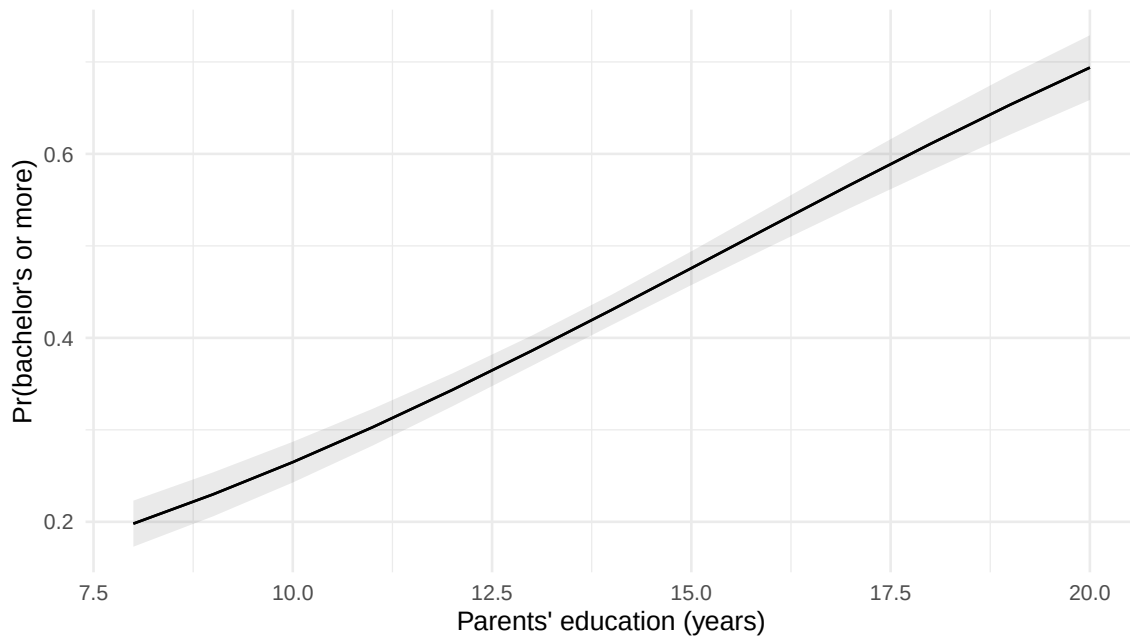


Figure 1: Average predicted probability of holding a BA by parents' education (observed-value approach), with 95% intervals.

i Note

`plot_predictions(m, by = "pareduc")` without the counterfactual grid is a different quantity: the average fitted probability *among respondents whose parents have that much education*. At 20 years it gives 0.75 rather than 0.69, and at 8 years 0.16 rather than 0.20, because those respondents also differ in vocabulary and the other covariates. The counterfactual version isolates parents' education; the subgroup version describes who those people are.

Contrast with setting everything else to its mean, which describes one hypothetical person (a 0.48 woman, a 0.15 Black respondent). Build that grid explicitly: `datagrid()`'s default is *not* the mean for every variable. It uses the mean for continuous covariates but the mode for binary and integer-valued ones, which here would give a man, not Black, with a vocabulary score of 6.

```
## one row per pareduc value, every other covariate at its sample mean
at_means <- datagrid(model = m_logit, pareduc = c(12, 16), wordsum = mean(gss$wordsum),
                     female = mean(gss$female), black = mean(gss$black))
predictions(m_logit, newdata = at_means)
#>
#>   pareduc wordsum female black Estimate Pr(>|z|)      S 2.5 % 97.5 %
#>      12     6.33  0.482 0.154    0.324  <0.001 196.1 0.305  0.343
#>      16     6.33  0.482 0.154    0.522  0.0737   3.8 0.498  0.546
#>
#> Type: invlink(link)
```

2.3 Then marginal effects: one number per variable

```
## AME: each person's own slope  $dp/dx = \beta * p(1 - p)$ , averaged
avg_slopes(m_logit)
#>
#>   Term Contrast Estimate Std. Error      z Pr(>|z|)      S 2.5 % 97.5 %
#> black      1 - 0 -0.0219   0.02097 -1.05   0.296   1.8 -0.0630 0.0192
#> female     1 - 0  0.0536   0.01488  3.60  <0.001  11.6  0.0245 0.0828
#> pareduc    dY/dX  0.0397   0.00226 17.57  <0.001 227.1  0.0353 0.0441
#> wordsum    dY/dX  0.0752   0.00395 19.03  <0.001 265.8  0.0675 0.0830
#>
#> Type: response
```

Compare with the marginal effect at the mean and with the LPM coefficients:

```
## MEM: the slope for one person at the sample means (same caution as above:
## newdata = "mean" would use modes for the binary and integer variables)
mem <- slopes(m_logit, newdata = datagrid(model = m_logit, pareduc = mean(gss$pareduc),
                                          wordsum = mean(gss$wordsum), female = mean(gss$female),
                                          black = mean(gss$black)))

ame <- avg_slopes(m_logit)
## match by variable name: avg_slopes() returns terms in alphabetical order,
## coef() in formula order
comparison <- data.frame(
  AME = ame$estimate,
  MEM = mem$estimate[match(ame$term, mem$term)],
  LPM = coef(m_lpm)[ame$term],
  row.names = ame$term
)
round(comparison, 4)
#>           AME      MEM      LPM
#> black -0.0219 -0.0267 -0.0379
```

```
#> female    0.0536  0.0656  0.0500
#> pareduc   0.0397  0.0487  0.0348
#> wordsum    0.0752  0.0922  0.0721
```

- The AME and the LPM coefficients are close. This is typical, and it is the main reason people defend the LPM for descriptive and causal work.
- The MEM answers a different question: the slope for one hypothetical person at the mean of everything, including the means of dummies.
- Report the AME (Hanmer and Kalkan 2013).

2.4 Odds ratios, at most as a footnote

```
round(exp(cbind(OR = coef(m_logit), confint.default(m_logit))), 3)
#>                OR 2.5 % 97.5 %
#> (Intercept) 0.003 0.002  0.005
#> pareduc      1.229 1.198  1.262
#> wordsum      1.479 1.411  1.550
#> female       1.321 1.135  1.538
#> black        0.892 0.719  1.106
```

Each correct word multiplies the *odds* of a BA by 1.48. That is one number for everyone, because the model is linear on the log-odds scale. It is not “48% more likely”: the probability change depends on where the person starts, which is why the probabilities came first.

2.5 Uncertainty: robust and clustered standard errors

Every quantity in this section, a predicted probability, a contrast, an AME, is a function $q(\hat{\beta})$ of the coefficients. Its standard error comes from the delta method: linearize q around $\hat{\beta}$ and use $\widehat{\text{Var}}(q) = \nabla q' \hat{V}(\hat{\beta}) \nabla q$. The gradient ∇q is fixed by the model; the only choice is which $\hat{V}(\hat{\beta})$ to plug in. The `vcov` argument of every `marginaleffects` function makes that choice, so a robust or clustered standard error for an AME is the same sandwich as in Week 2, pushed through the same gradient.

Three variance matrices for the coefficients first:

```
## classical: inverse information (Week 3). HC0: the sandwich, each person's own
## squared score. HC1: HC0 times n/(n - k). cluster: scores summed within a GSS
## sampling unit (psu) before squaring, so people in the same psu may be correlated
round(cbind(
  classical = sqrt(diag(vcov(m_logit))),
  HC0       = sqrt(diag(vcovHC(m_logit, type = "HC0"))),
  HC1       = sqrt(diag(vcovHC(m_logit, type = "HC1"))),
  cluster   = sqrt(diag(vcovCL(m_logit, cluster = ~ psu)))
), 4)
#>                classical      HC0      HC1 cluster
```

```
#> (Intercept)    0.2454 0.2662 0.2664 0.2697
#> pareduc       0.0134 0.0155 0.0155 0.0159
#> wordsum       0.0240 0.0247 0.0247 0.0246
#> female        0.0776 0.0771 0.0772 0.0734
#> black         0.1098 0.1098 0.1099 0.1097
```

The same four choices carried through to the AMEs. `vcov = TRUE` is the classical matrix, a string names an HC type, a one-sided formula names the cluster variable:

```
se_of <- function(v) {
  a <- as.data.frame(avg_slopes(m_logit, vcov = v))
  setNames(a$std.error, a$term)
}
round(rbind(classical = se_of(TRUE), HC0 = se_of("HC0"), HC1 = se_of("HC1"),
            cluster_psu = se_of(~ psu)), 4)
#>          black female pareduc wordsum
#> classical  0.0210 0.0149 0.0023 0.0040
#> HC0        0.0210 0.0148 0.0026 0.0041
#> HC1        0.0210 0.0148 0.0026 0.0041
#> cluster_psu 0.0209 0.0140 0.0027 0.0041
```

Predicted probabilities take the same argument:

```
avg_predictions(m_logit, variables = list(pareduc = c(12, 16)), vcov = ~ psu)
#>
#>   pareduc Estimate Std. Error    z Pr(>|z|)      S 2.5 % 97.5 %
#>      12    0.343    0.0102 33.6  <0.001 819.5 0.323 0.363
#>      16    0.521    0.0123 42.5  <0.001   Inf 0.497 0.545
#>
#> Type: response
```

Two things to notice. The robust and classical standard errors are within a few percent of each other here: for a logit, a large gap between them is a warning that the model is misspecified rather than a fix, because the logit's variance is not a free parameter the way the Poisson's is. And clustering by `psu` matters little for this outcome because the GSS sampling units are small (839 of them, a median of four respondents each); with respondents nested in states or years it would matter more. A full design-based analysis would also use the strata and weights, through the `survey` package.

2.6 Probit: another S-curve

```
## coefficients differ by a scale factor; AMEs do not
round(cbind(logit = coef(m_logit), probit = coef(m_probit),
            ratio = coef(m_logit) / coef(m_probit)), 3)
```

```

#>           logit probit ratio
#> (Intercept) -5.810 -3.367 1.726
#> pareduc      0.207  0.117 1.768
#> wordsum      0.391  0.233 1.678
#> female       0.279  0.159 1.751
#> black        -0.114 -0.070 1.643
round(cbind(AME_logit = avg_slopes(m_logit)$estimate,
            AME_probit = avg_slopes(m_probit)$estimate), 4) |>
  `rownames<-`(avg_slopes(m_logit)$term)
#>           AME_logit AME_probit
#> black      -0.0219   -0.0225
#> female      0.0536    0.0515
#> pareduc     0.0397    0.0378
#> wordsum     0.0752    0.0754

```

The coefficients differ by a factor of about 1.6 to 1.7 because the two S-curves have different steepness at the center. The AMEs agree to two decimals. Default to logit; use probit when the next step needs a normal (selection models, bivariate probit) or when the subfield expects it.

! Important

Never compare raw logit or probit coefficients across nested models or across groups. Adding a covariate rescales every coefficient, even ones whose effect is unchanged (Mood 2010). Compare AMEs or predicted probabilities.

3 Interactions

Does parents' education matter more for women? Two questions: should the product `pareduc * female` be in the model, and do the models with and without it agree on the probability scale?

```

## no product: one log-odds slope for pareduc, forced equal for men and women
m_np <- glm(ba ~ pareduc + female + wordsum + black, data = gss, family = binomial)
## with product: the slope is beta_1 + beta_12 * female; the data choose beta_12
m_int <- glm(ba ~ pareduc * female + wordsum + black, data = gss, family = binomial)
round(coef(summary(m_int))[, 1:3], 3)
#>           Estimate Std. Error z value
#> (Intercept)   -5.675      0.297  -19.082
#> pareduc        0.197      0.018   10.857
#> female       -0.012      0.377   -0.031
#> wordsum        0.391      0.024   16.286
#> black         -0.116      0.110   -1.055
#> pareduc:female  0.021      0.027    0.786
## LR test of the product

```

```
anova(m_np, m_int, test = "LRT")
#> Analysis of Deviance Table
#>
#> Model 1: ba ~ pareduc + female + wordsum + black
#> Model 2: ba ~ pareduc * female + wordsum + black
#>   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
#> 1       3504       3959
#> 2       3503       3958  1    0.619    0.43
```

The product is 0.021 on the log-odds scale, $z = 0.79$. That is the wrong scale for the question. The AME of parents' education, by gender, in both models:

```
## no product: the AMEs still differ by gender, through  $p(1 - p)$  alone
avg_slopes(m_np, variables = "pareduc", by = "female")
#>
#>   female Estimate Std. Error    z Pr(>|z|)    S 2.5 % 97.5 %
#>     0    0.0386    0.00220 17.6  <0.001 227.4 0.0343 0.0429
#>     1    0.0409    0.00234 17.4  <0.001 223.8 0.0363 0.0455
#>
#> Term: pareduc
#> Type: response
#> Comparison: dY/dX
## with product: they differ through  $p(1 - p)$  and through  $\beta_{12}$ 
avg_slopes(m_int, variables = "pareduc", by = "female")
#>
#>   female Estimate Std. Error    z Pr(>|z|)    S 2.5 % 97.5 %
#>     0    0.0371    0.00296 12.5  <0.001 116.8 0.0313 0.0429
#>     1    0.0428    0.00335 12.8  <0.001 121.8 0.0362 0.0493
#>
#> Term: pareduc
#> Type: response
#> Comparison: dY/dX
```

```
## the interaction we report: the difference in AMEs, with an interval
avg_slopes(m_int, variables = "pareduc", by = "female",
            hypothesis = "b2 - b1 = 0")
#>
#> Hypothesis Estimate Std. Error    z Pr(>|z|)    S 2.5 % 97.5 %
#>   b2-b1=0    0.00571    0.00439 1.3    0.194  2.4 -0.0029 0.0143
#>
#> Type: response
```

- **Include the product** whenever the question is whether X_1 's effect differs by X_2 . Without it the answer is fixed in advance: zero on the log-odds scale, and on the probability scale

whatever the S-curve gives (the no-product model already shows women with a larger AME, because they sit where the curve is steeper: Ai and Norton 2003).

- **The two models agree** on probabilities only when $\hat{\beta}_{12}$ is near zero, as here. When it is not, the no-product model has the wrong probabilities in the cells.
- **Report** the difference in AMEs with its interval, here 0.006 per year, SE 0.004. The answer is no.

4 Outcomes with several categories

4.1 Multinomial logit: a nominal outcome

Where a respondent lives, in four census regions. The categories have no order, so one logit for every region against a base (the Midwest).

```
## collapse the nine GSS regions to four; Midwest first so it is the base
gss$region4 <- factor(case_when(
  gss$region %in% c("New England", "Middle Atlantic") ~ "Northeast",
  gss$region %in% c("E. North Central", "W. North Central") ~ "Midwest",
  gss$region %in% c("South Atlantic", "E. South Central", "W. South Central") ~ "South",
  TRUE ~ "West"), levels = c("Midwest", "Northeast", "South", "West"))
table(gss$region4)
#>
#>   Midwest Northeast      South      West
#>      861      590     1298      760

m_mn <- multinom(region4 ~ educ + black + female + age, data = gss, trace = FALSE)
## one row per non-base region: exp(coef) multiplies the odds AGAINST THE MIDWEST
round(exp(coef(m_mn)), 2)
#>           (Intercept) educ black female  age
#> Northeast          0.21 1.08  1.32   1.08 1.00
#> South              1.52 0.98  2.88   1.21 1.00
#> West                0.99 1.01  0.82   1.13 0.99
```

Read the **black** column: being Black multiplies the odds of South-versus-Midwest by about 2.9 and of West-versus-Midwest by about 0.8. Those are odds against the base, not probabilities; the four probabilities move together and add to one:

```
## observed-value predicted probabilities, one column per region
avg_predictions(m_mn, variables = "black")
#>
#>      Group black Estimate Std. Error      z Pr(>|z|)      S 2.5 % 97.5 %
#> Midwest      0    0.261    0.00806 32.38 <0.001 761.6 0.2451 0.277
#> Midwest      1    0.160    0.01591 10.03 <0.001  76.3 0.1284 0.191
#> Northeast    0    0.173    0.00692 25.03 <0.001 456.9 0.1596 0.187
#> Northeast    1    0.141    0.01517  9.29 <0.001  65.8 0.1112 0.171
```

```
#> South      0      0.331      0.00863 38.36 <0.001 1066.8 0.3142 0.348
#> South      1      0.582      0.02132 27.28 <0.001  542.1 0.5398 0.623
#> West       0      0.235      0.00779 30.15 <0.001  661.1 0.2196 0.250
#> West       1      0.118      0.01389  8.49 <0.001   55.4 0.0907 0.145
#>
#> Type: probs
## check: for each person the four probabilities sum to one
range(rowSums(fitted(m_mn)))
#> [1] 1 1
```

```
## do the covariates matter at all? LR against the intercept-only model
m_mn0 <- multinom(region4 ~ 1, data = gss, trace = FALSE)
lr <- 2 * (as.numeric(logLik(m_mn)) - as.numeric(logLik(m_mn0)))
df <- length(coef(m_mn)) - length(coef(m_mn0))
c(LR = round(lr, 1), df = df)
#> LR df
#> 171.2 12.0
pchisq(lr, df, lower.tail = FALSE) < 0.001
#> [1] TRUE
```

4.2 Ordered logit: an ordered outcome

Highest degree, five ordered categories. One coefficient vector and four cutpoints.

```
m_ord <- polr(degree ~ pareduc + wordsum + female + black,
              data = gss, Hess = TRUE)
round(coef(summary(m_ord)), 4)
#>
#> Value Std. Error t value
#> pareduc      0.2082      0.0104 19.9296
#> wordsum       0.3578      0.0192 18.5908
#> female        0.3349      0.0646  5.1829
#> black        -0.0238      0.0914 -0.2607
#> < HS|HS       1.8372      0.1616 11.3667
#> HS|Junior college 5.2081      0.1788 29.1209
#> Junior college|Bachelor 5.6724      0.1823 31.1134
#> Bachelor|Graduate 7.3582      0.1973 37.3032
```

`polr()` does not print *p*-values; get them from the *t*-values, or better, work on the probability scale.

```
## observed-value predictions: set wordsum to 3 for everyone, then 9, keep the
## other covariates as observed, average the five probabilities
avg_predictions(m_ord, variables = list(wordsum = c(3, 9)), type = "probs")
#>
#> Group wordsum Estimate Std. Error z Pr(>|z|) S 2.5 % 97.5 %
```

```

#> < HS          3  0.1265    0.00888 14.25    <0.001 150.7 0.1091 0.1439
#> < HS          9  0.0190    0.00195  9.72    <0.001  71.8 0.0152 0.0228
#> HS           3  0.6228    0.01180 52.80    <0.001   Inf 0.5997 0.6459
#> HS           9  0.2839    0.01091 26.02    <0.001 493.4 0.2625 0.3053
#> Junior college 3  0.0719    0.00446 16.12    <0.001 191.8 0.0632 0.0807
#> Junior college 9  0.0936    0.00506 18.49    <0.001 251.2 0.0837 0.1035
#> Bachelor       3  0.1373    0.00812 16.90    <0.001 210.5 0.1214 0.1532
#> Bachelor       9  0.3534    0.00993 35.58    <0.001 918.6 0.3339 0.3729
#> Graduate       3  0.0414    0.00367 11.27    <0.001  95.5 0.0342 0.0486
#> Graduate       9  0.2501    0.01092 22.91    <0.001 383.5 0.2287 0.2715
#>
#> Type: probs

```

Four more words move about a quarter of the population out of “high school only,” mostly into the two degree categories. The middle category barely moves: one coefficient pushes probability *through* a narrow band without piling it up there, so the sign of the effect on a middle category is not the sign of $\hat{\beta}$.

```

## AMEs: one per covariate per category, the average change in each category's
## probability per unit of the covariate (or for the dummy switching 0 to 1)
avg_slopes(m_ord, type = "probs")
#>
#>      Term          Group Contrast  Estimate Std. Error      z Pr(>|z|)      S
#> black < HS          1 - 0  0.001164   0.004492   0.259   0.795   0.3
#> black HS           1 - 0  0.003597   0.013779   0.261   0.794   0.3
#> black Junior college 1 - 0 -0.000127   0.000506  -0.250   0.803   0.3
#> black Bachelor      1 - 0 -0.002080   0.008012  -0.260   0.795   0.3
#> black Graduate      1 - 0 -0.002555   0.009753  -0.262   0.793   0.3
#> female < HS         1 - 0 -0.016150   0.003193  -5.057  <0.001  21.2
#> female HS          1 - 0 -0.050883   0.009855  -5.163  <0.001  22.0
#> female Junior college 1 - 0  0.001727   0.000464   3.723  <0.001  12.3
#> female Bachelor    1 - 0  0.029201   0.005701   5.122  <0.001  21.7
#> female Graduate    1 - 0  0.036105   0.007015   5.147  <0.001  21.9
#> pareduc < HS       dY/dX -0.010120   0.000702 -14.414  <0.001 154.1
#> pareduc HS         dY/dX -0.031463   0.001510 -20.840  <0.001 318.0
#> pareduc Junior college dY/dX 0.001062   0.000207   5.141  <0.001  21.8
#> pareduc Bachelor    dY/dX 0.018093   0.000970  18.645  <0.001 255.3
#> pareduc Graduate    dY/dX 0.022428   0.001223  18.336  <0.001 247.0
#> wordsum < HS       dY/dX -0.017389   0.001283 -13.554  <0.001 136.6
#> wordsum HS         dY/dX -0.054063   0.002695 -20.059  <0.001 294.9
#> wordsum Junior college dY/dX 0.001825   0.000356   5.132  <0.001  21.7
#> wordsum Bachelor    dY/dX 0.031089   0.001716  18.119  <0.001 241.3
#> wordsum Graduate    dY/dX 0.038539   0.002238  17.219  <0.001 218.3

```

```

#>      2.5 %    97.5 %
#> -0.007640  0.009968
#> -0.023409  0.030602
#> -0.001118  0.000865
#> -0.017783  0.013624
#> -0.021670  0.016561
#> -0.022409 -0.009891
#> -0.070198 -0.031568
#>  0.000818  0.002636
#>  0.018027  0.040375
#>  0.022357  0.049854
#> -0.011496 -0.008744
#> -0.034422 -0.028504
#>  0.000657  0.001467
#>  0.016191  0.019995
#>  0.020031  0.024826
#> -0.019904 -0.014875
#> -0.059346 -0.048781
#>  0.001128  0.002522
#>  0.027726  0.034452
#>  0.034152  0.042925
#>
#> Type: probs

```

Read the `group` column: each covariate has five AMEs, one per category, and the five sum to zero because the probabilities sum to one. This is the standard way to report an ordered logit: category probabilities at chosen values, and the AMEs by category, rather than the cutpoints.

5 Count outcomes

Outcome: `childs`, the number of children.

```

f_c <- childs ~ educ + female + black + exper + evermar
m_pois <- glm(f_c, data = gss, family = poisson(link = "log"))
## exp(coef) multiplies the expected count
round(cbind(coef(summary(m_pois))[, 1:2], exp_b = exp(coef(m_pois))), 3)
#>           Estimate Std. Error exp_b
#> (Intercept)  -0.006      0.088 0.994
#> educ         -0.045      0.005 0.956
#> female        0.009      0.027 1.009
#> black         0.360      0.035 1.433
#> exper         0.012      0.001 1.012
#> evermar       0.947      0.043 2.578

```

5.1 Is it overdispersed?

The Poisson makes one strong claim: $\text{Var}(Y_i | X_i) = \theta_i$. Check it three ways.

```
## raw: variance against mean
c(mean = mean(gss$childs), var = var(gss$childs))
#> mean    var
#> 1.604 2.109

## Pearson dispersion: under the Poisson each  $(y - \theta)^2 / \theta$  averages 1
n <- nrow(gss); k <- length(coef(m_pois))
phi <- sum((gss$childs - fitted(m_pois))^2 / fitted(m_pois)) / (n - k)
c(phi = phi)
#> phi
#> 1.149

## formal test, using only the Poisson fit. trafo says what shape the extra
## variance takes: trafo = 1 tests  $\text{Var} = \theta + \alpha * \theta$  (a constant
## multiple of the mean, the quasi-Poisson shape); trafo = 2 tests
##  $\text{Var} = \theta + \alpha * \theta^2$  (the negative-binomial shape). Either way
##  $H_0$  is  $\alpha = 0$  and the printed alpha is the estimated excess
dispersiontest(m_pois, trafo = 1)
#>
#> Overdispersion test
#>
#> data:  m_pois
#> z = 4, p-value = 0.00004
#> alternative hypothesis: true alpha is greater than 0
#> sample estimates:
#> alpha
#> 0.1676
```

The counts scatter 15% more than the Poisson allows. The claim is false, so the Poisson SE is wrong. Three repairs, each replacing the claim with a different one:

```
m_qp <- glm(f_c, data = gss, family = quasipoisson) # Var = phi * theta
m_nb <- glm.nb(f_c, data = gss) # Var = theta + theta^2 / alpha

## the coefficients barely move across the three likelihoods ...
round(cbind(poisson = coef(m_pois), quasi = coef(m_qp), negbin = coef(m_nb)), 3)
#>           poisson  quasi negbin
#> (Intercept) -0.006 -0.006 -0.005
#> educ        -0.045 -0.045 -0.046
#> female       0.009  0.009  0.011
#> black       0.360  0.360  0.366
```

```

#> exper      0.012  0.012  0.012
#> evermar    0.947  0.947  0.950

## ... the standard errors do
round(cbind(
  poisson = sqrt(diag(vcov(m_pois))),
  quasi   = sqrt(diag(vcov(m_qp))),
  negbin   = sqrt(diag(vcov(m_nb)))[names(coef(m_pois))],
  robust   = sqrt(diag(vcovHC(m_pois, type = "HC0"))) # no claim at all
), 4)
#>           poisson  quasi negbin robust
#> (Intercept) 0.0876 0.0939 0.0900 0.1012
#> educ        0.0048 0.0051 0.0049 0.0052
#> female      0.0269 0.0288 0.0277 0.0278
#> black       0.0349 0.0374 0.0361 0.0373
#> exper       0.0013 0.0014 0.0014 0.0013
#> evermar     0.0430 0.0461 0.0437 0.0551

## the quasi-Poisson SE is the Poisson SE times sqrt(phi); check on evermar
c(by_hand = sqrt(diag(vcov(m_pois)))["evermar"] * sqrt(phi),
  quasi    = sqrt(diag(vcov(m_qp)))["evermar"])
#> by_hand.evermar  quasi.evermar
#>      0.04605      0.04605

```

Quasi-Poisson keeps the Poisson coefficients exactly, since it changes only the variance; the negative binomial refits and lands within a hundredth of them. The standard errors are what move. The robust SE for `evermar` is much larger than the other two because the excess scatter is not uniform: it sits among the never married, exactly the group this dummy compares.

5.2 Excess zeros, and the hurdle model

```

## share of zeros: observed, and averaged over the fitted distributions
c(observed = mean(gss$chlds == 0),
  poisson  = mean(dpois(0, fitted(m_pois))),
  negbin    = mean(dnbinom(0, mu = fitted(m_nb), size = m_nb$theta)))
#> observed  poisson  negbin
#>  0.3007   0.2519   0.2592

```

Thirty percent are childless; neither model has a way to make zeros special. The hurdle model does: a logit for zero versus positive, then a truncated Poisson for the positives, the same covariates in both parts.

```

m_hur <- hurdle(f_c, data = gss, dist = "poisson")
## the two parts tell different stories

```

```

round(cbind(any_child = exp(coef(m_hur, model = "zero")),
        how_many = exp(coef(m_hur, model = "count"))), 2)
#>           any_child how_many
#> (Intercept)      1.08      2.09
#> educ          0.89      0.96
#> female        1.34      0.93
#> black         2.92      1.25
#> exper         1.03      1.01
#> evermar       8.91      1.35
## the zero share is matched exactly: the logit part is fitted to it
mean(predict(m_hur, type = "prob")[, 1])
#> [1] 0.3007

```

Marriage multiplies the odds of *any* child by about 9 but the count among parents by about 1.35; women are likelier to have any child and have slightly fewer. One Poisson averaged these two processes.

The zero-inflated model tells the other story: a share of people are “structural zeros” who will never have a child, and everyone else follows a Poisson that can also produce zeros. Same syntax, different meaning of the zero part.

```

m_zip <- zeroinfl(f_c, data = gss, dist = "poisson")
## zero part: a logit for being a STRUCTURAL zero (not for having any child, as
## in the hurdle), so the signs flip relative to the hurdle's zero part
round(cbind(structural_zero = exp(coef(m_zip, model = "zero")),
        count_part = exp(coef(m_zip, model = "count"))), 2)
#>           structural_zero count_part
#> (Intercept)          0.01      1.70
#> educ          1.47      0.97
#> female        0.33      0.96
#> black         0.18      1.24
#> exper         0.99      1.01
#> evermar       0.00      1.25
## two kinds of zero: the structural share, and the total share the model implies
c(structural = mean(predict(m_zip, type = "zero")),
  total      = mean(predict(m_zip, type = "prob")[, 1]),
  observed   = mean(gss$chlds == 0))
#> structural      total      observed
#>      0.1556      0.2908      0.3007

```

Read the zero part with care: an odds ratio of 0.003 for **evermar** says the ever-married are almost never structural zeros; the structural zeros are the never-married, and the more educated. About 16% of respondents are estimated to be structural zeros; the rest of the 30% childless are “zeros for now” from the count part. Unlike the hurdle, the zero share is not matched exactly.

5.3 Comparing the count models by AIC and BIC

Poisson and negative binomial are nested (the Poisson is the negative binomial with $\alpha \rightarrow \infty$), so an LR test applies. The hurdle and the zero-inflated model are not nested in either, or in each other, so use AIC and BIC, which need only a log-likelihood and a parameter count.

```
## LR: negative binomial against Poisson, one parameter (alpha)
2 * (as.numeric(logLik(m_nb)) - as.numeric(logLik(m_pois)))
#> [1] 4.805

## AIC = -2 loglik + 2k; BIC = -2 loglik + k log n
mods <- list(Poisson = m_pois, `Neg. binomial` = m_nb, Hurdle = m_hur, `Zero-inflated` = m_zi)
tab <- t(sapply(mods, function(m) {
  ll <- logLik(m)
  c(logLik = as.numeric(ll), k = attr(ll, "df"),
    AIC = -2 * as.numeric(ll) + 2 * attr(ll, "df"),
    BIC = -2 * as.numeric(ll) + attr(ll, "df") * log(n))
}))
round(tab, 1)
#>           logLik  k   AIC   BIC
#> Poisson      -5383  6 10778 10815
#> Neg. binomial -5380  7 10775 10818
#> Hurdle       -5228 12 10479 10553
#> Zero-inflated -5200 12 10423 10497
## check against the built-in functions
round(c(AIC(m_hur), BIC(m_hur)), 1)
#> [1] 10479 10553
```

Read the Poisson-versus-hurdle line the Week 3 way. The hurdle adds six parameters (a second intercept and five more coefficients). BIC charges $6 \log 3509 = 49$ for them, AIC charges 12; the log-likelihood rises by about 155, so -2ℓ falls by about 310. Both criteria prefer the hurdle by a wide margin, and BIC's heavier penalty does not change the verdict. The zero-inflated model, with the same twelve parameters, fits better still, by about 56 on either criterion. Fit alone would pick it; whether a permanently childless class is the right story is a substantive question the criteria cannot answer, and the hurdle's "any child, then how many" reading is the one demographers usually defend. The negative binomial is the interesting line: one extra parameter, a gain of 2.4 in log-likelihood. AIC prefers it to the Poisson by about 3; BIC prefers the Poisson by about 3. That is Week 3's disagreement in the wild: the evidence for α is real (the LR is 4.8) but not strong enough to pay BIC's $\log 3509 = 8.2$ per parameter. Both criteria agree that the zeros, not the variance, are where the Poisson is wrong.

5.4 Does the model reproduce the data?

```
## for each count k, average Pr(Y = k) over everyone's fitted distribution
ks <- 0:8
dist <- bind_rows(
  tibble::tibble(k = ks, source = "observed",
    p = as.numeric(table(factor(gss$chlds, levels = ks))) / n),
  tibble::tibble(k = ks, source = "Poisson",
    p = sapply(ks, function(j) mean(dpois(j, fitted(m_pois))))),
  tibble::tibble(k = ks, source = "Neg. binomial",
    p = sapply(ks, function(j) mean(dnbinom(j, mu = fitted(m_nb), size = m_nb$th))),
  tibble::tibble(k = ks, source = "Hurdle",
    p = colMeans(predict(m_hur, type = "prob")[, 1:9]))
) |> mutate(source = factor(source, levels = c("observed", "Poisson", "Neg. binomial", "Hurdle")))

ggplot(dist, aes(k, p, fill = source)) +
  geom_col(position = "dodge") +
  scale_fill_manual(values = c("observed" = "grey30", "Poisson" = "navy",
    "Neg. binomial" = "firebrick", "Hurdle" = "darkgreen")) +
  labs(x = "number of children", y = "proportion", fill = NULL) +
  theme_minimal(base_size = 10)
```

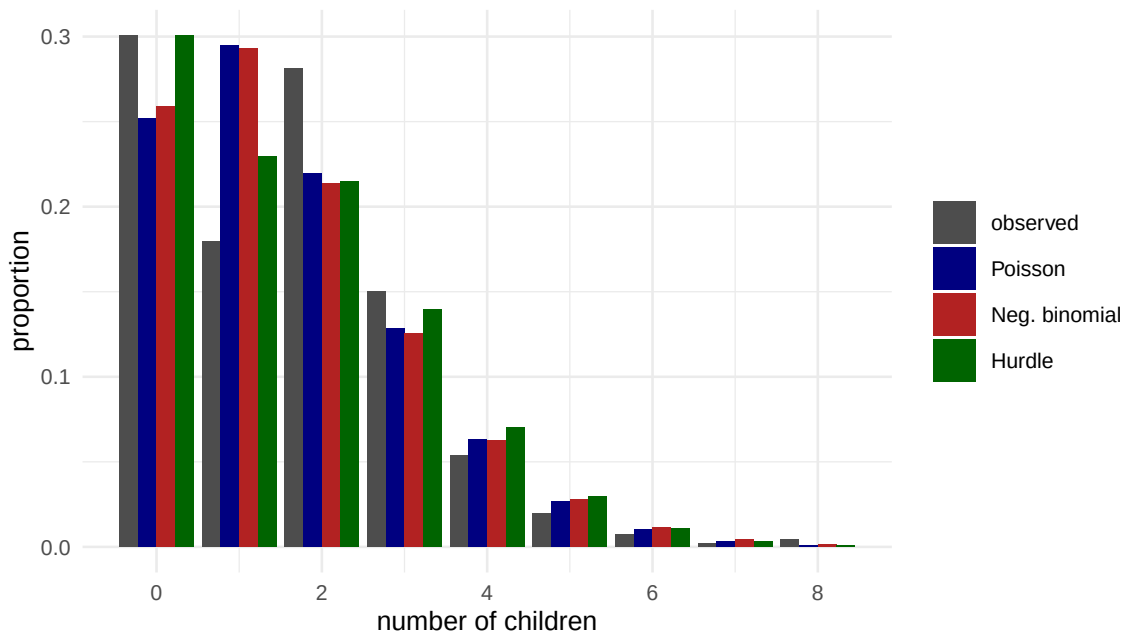


Figure 2: Observed distribution of the number of children against the distributions implied by the Poisson, negative binomial, and hurdle fits.

This predictive check is the single most informative count diagnostic, and the one people most often skip. The hurdle matches the zeros by construction; no count model matches the bump at

two children.

i Note

Which count model. Start with Poisson and robust SEs. Run the overdispersion test; if it rejects and you need a likelihood (prediction, AIC, an LR test), fit the negative binomial. Look at the observed and predicted distributions. Choose hurdle or zero-inflated on substance: hurdle when crossing from zero to one is a different *process*, zero-inflated when some zeros are a different *kind* of person. The two are not nested; AIC or BIC between them.

6 Multilevel models

The GSS extract has no natural grouping, so this section uses the High School and Beyond data that ship with `nlme`: 7,185 students in 160 schools, a math achievement score, student SES, and school sector.

```
data(MathAchieve, package = "nlme"); data(MathAchSchool, package = "nlme")
hsb <- merge(as.data.frame(MathAchieve),
             as.data.frame(MathAchSchool)[, c("School", "Sector")], by = "School")
hsb$School <- factor(as.character(hsb$School)) # plain factor, not ordered
c(students = nrow(hsb), schools = length(unique(hsb$School)))
#> students schools
#>      7185      160
range(table(hsb$School))
#> [1] 14 67
```

6.1 Reading the lmer formula

The random-intercept model is

$$\text{MathAch}_{ij} = \beta_0 + \beta_1 \text{SES}_{ij} + \beta_2 \text{Catholic}_{ij} + u_j + e_{ij}, \quad u_j \sim N(0, \tau^2), \quad e_{ij} \sim N(0, \sigma^2).$$

In `lmer` the formula is `MathAch ~ SES + Sector + (1 | School)`. The part outside the parentheses is the fixed part, an ordinary regression formula for the β s. The part in parentheses is the random part, and it has two sides. Right of the bar is the grouping variable: which rows share a u_j . Left of the bar is the design of the random part: which columns get a coefficient that varies by group. 1 is the intercept column, so `(1 | School)` gives every school its own intercept shift, the u_j that multiplies the column of ones; the model estimates its variance τ^2 , not the 160 shifts.

```
(1 | School)          # random intercept: u_j, one variance tau^2
(1 + SES | School)    # random intercept AND a random slope on SES:
                      #   u_j + v_j * SES_ij, two variances and their covariance
```

```
(0 + SES | School)      # random slope only, common intercept
(1 | School) + (1 | District)  # two grouping levels, one u for each
```

A random slope says the SES gradient itself differs across schools; exercise 7 fits it. Everything in this lab keeps (1 | School).

6.2 Predictors at both levels

```
m_ols <- lm(MathAch ~ SES + Sector, data = hsb)
## REML = FALSE: plain maximum likelihood, so logLik() is the likelihood from
## Week 3 and models with different fixed effects can be compared by LR, AIC,
## BIC. The default REML = TRUE corrects the variances for the degrees of
## freedom spent on the betas (like RSS/(n - k) in OLS) but its likelihood is
## not comparable across fixed effects. On HSB the two differ in the third decimal
m_re <- lmer(MathAch ~ SES + Sector + (1 | School), data = hsb, REML = FALSE)
## fixed effects: school dummies first. Sector is constant within school, so
## it is collinear with the dummies; lm() drops the LAST collinear column, and
## with Sector listed before School it would keep Sector and drop a school
## dummy instead, printing a sector "effect" measured against whichever school
## was dropped. Listing School first makes lm() report the honest NA.
m_fe <- lm(MathAch ~ School + SES + Sector, data = hsb)

round(cbind(
  pooled      = coef(m_ols)[c("SES", "SectorCatholic")],
  pooled_clSE = sqrt(diag(vcovCL(m_ols, cluster = hsb$School)))[c("SES", "SectorCatholic")],
  random_int  = fixef(m_re)[c("SES", "SectorCatholic")],
  random_SE   = sqrt(diag(vcov(m_re)))[c("SES", "SectorCatholic")],
  fixed_eff   = coef(m_fe)[c("SES", "SectorCatholic")]), 3)
#>
#>      pooled pooled_clSE random_int random_SE fixed_eff
#> SES      2.949      0.128      2.377      0.105      2.191
#> SectorCatholic 1.935      0.317      2.100      0.339      NA
```

Sector is NA under fixed effects: it is constant within school, so the school dummies absorb it. The random-intercept model estimates it, with a standard error that treats the 160 schools, not the 7,185 students, as the units. The SES slope sits between the pooled and the fixed-effects one: the random-intercept model uses both within-school and between-school variation.

6.3 The two variances and the intraclass correlation

```
## tau2 = between-school variance of the intercepts, sigma2 = within-school
vc <- as.data.frame(VarCorr(m_re))
tau2 <- vc$vcov[1]; sigma2 <- vc$vcov[2]
c(tau2 = tau2, sigma2 = sigma2, ICC = tau2 / (tau2 + sigma2))
```

```
#>      tau2      sigma2      ICC
#> 3.62187 37.03279 0.08909

## is tau2 zero? LR against OLS with the same x, one parameter pinned at zero
2 * (as.numeric(logLik(m_re)) - as.numeric(logLik(m_ols)))
#> [1] 332.2
```

Of the variation SES and sector leave unexplained, 9% is between schools and 91% within. The LR statistic is enormous; schools still differ after the controls.

6.4 A random-intercept logit

```
hsb$top <- as.integer(hsb$MathAch > quantile(hsb$MathAch, 0.75))
g1 <- glmer(top ~ SES + Sector + (1 | School), data = hsb,
            family = binomial, nAGQ = 10) # nAGQ: accuracy of the integral over u_j
round(summary(g1)$coefficients, 3)
#>
#> Estimate Std. Error z value Pr(>|z|)
#> (Intercept) -1.460 0.077 -18.858 0
#> SES 0.674 0.045 14.979 0
#> SectorCatholic 0.391 0.112 3.494 0

## observed-value predictions: set every student's SES to each value, keep their
## own sector and their own school's predicted u_j (the default, re.form = NULL),
## average the 7,185 probabilities. re.form = NA would set every u_j to zero.
## The standard error carries the uncertainty in the coefficients only
avg_predictions(g1, variables = list(SES = seq(-2, 2, by = 0.5)))
#>
#> SES Estimate Std. Error z Pr(>|z|) S 2.5 % 97.5 %
#> -2.0 0.0757 0.00778 9.73 <0.001 71.8 0.0605 0.091
#> -1.5 0.1023 0.00847 12.08 <0.001 109.2 0.0857 0.119
#> -1.0 0.1365 0.00887 15.39 <0.001 175.0 0.1191 0.154
#> -0.5 0.1796 0.00913 19.68 <0.001 284.0 0.1617 0.198
#> 0.0 0.2323 0.00975 23.82 <0.001 414.3 0.2132 0.251
#> 0.5 0.2944 0.01155 25.48 <0.001 473.4 0.2718 0.317
#> 1.0 0.3648 0.01488 24.52 <0.001 438.8 0.3357 0.394
#> 1.5 0.4412 0.01920 22.98 <0.001 385.8 0.4036 0.479
#> 2.0 0.5204 0.02359 22.06 <0.001 355.7 0.4741 0.567
#>
#> Type: response
```

The same logit as this morning, with a school shift on the log-odds scale whose variance is estimated. A coefficient is the log-odds change holding the school's u_j fixed. The predictions put the coefficient on the probability scale: moving every student from SES -1 to SES $+1$ (the

SD of SES is 0.78) raises the share in the top quartile from 0.14 to 0.37.

The figure from the slides shows what the school shift does to the curve. For a public school, the S-curve depends on which school: $u_j = 0$ is the typical school, $\pm\hat{\tau}$ are schools one SD above and below. The population-average curve integrates the probability over $u_j \sim N(0, \hat{\tau}^2)$; averaging S-curves with different intercepts spreads the transition out, so it is flatter than the $u_j = 0$ curve, only slightly here because $\hat{\tau} = 0.58$ is small.

```
b <- fixef(g1)
tau <- sqrt(as.data.frame(VarCorr(g1))$vcov[1]) # SD of the school shifts
ses <- seq(-2, 2, by = 0.05)
eta <- b["(Intercept)"] + b["SES"] * ses # public school: Sector dummy = 0

## population average: integrate plogis(eta + u) over u ~ N(0, tau^2) on a grid
u <- seq(-4 * tau, 4 * tau, length.out = 401)
w <- dnorm(u, 0, tau); w <- w / sum(w)
p_avg <- sapply(eta, function(e) sum(plogis(e + u) * w))

curves <- data.frame(
  SES = rep(ses, 4),
  p = c(plogis(eta + tau), plogis(eta), plogis(eta - tau), p_avg),
  school = rep(c("u_j = +tau", "u_j = 0", "u_j = -tau", "average over schools"),
    each = length(ses)))
curves$school <- factor(curves$school, levels = unique(curves$school))
ggplot(curves, aes(SES, p, colour = school, linetype = school)) +
  geom_line(linewidth = 0.8) +
  scale_colour_manual(values = c("#00539B", "#00539B", "#00539B", "#C84E00")) +
  scale_linetype_manual(values = c("dashed", "solid", "dotted", "solid")) +
  labs(x = "Student SES", y = "Pr(top quartile in math)",
    colour = NULL, linetype = NULL) +
  theme_minimal(base_size = 10) + theme(legend.position = "top")
```

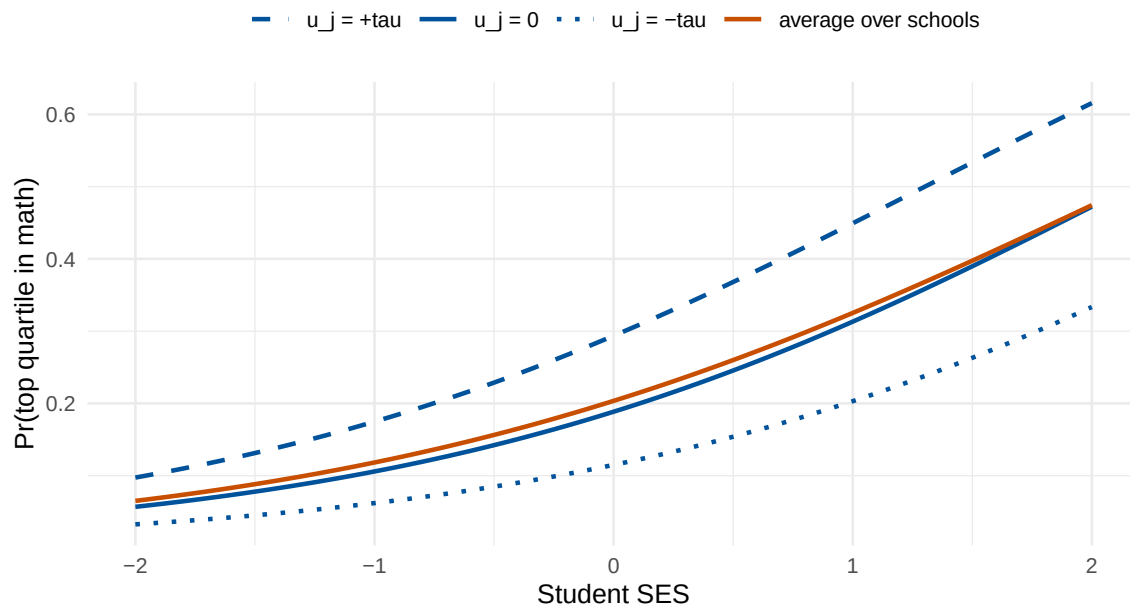


Figure 3: $\Pr(\text{top quartile in math})$ by SES, public schools: the curve for a typical school, for schools one SD above and below, and the average over the fitted distribution of school shifts.

The table above averages over the students as they are, in both sectors and in their own schools; the orange curve averages over the school distribution for public schools only. Both are population-average quantities; the coefficient 0.674 describes the blue curves, one school at a time.

7 Exercises

1. **Logit versus LPM for a causal contrast.** Estimate the “effect” of being ever-married (`evermar`) on holding a BA, controlling for `pareduc`, `wordsum`, `female`, `black`. Compare the LPM coefficient with the logit AME. How close are they? Which would you report, and to whom?
2. **The rescaling problem.** Fit the logit for `ba` with (a) `pareduc` only and (b) `pareduc` + `wordsum` + `female` + `black`. Compare the `pareduc` coefficient across the two, then compare the `pareduc` AME. Explain the difference using Mood (2010).
3. **Simulation-based intervals.** Take the logit model and get an interval for the difference in predicted probability between `pareduc` = 12 and `pareduc` = 16 by simulating 10,000 draws of $\tilde{\beta}$ from $\mathcal{N}(\hat{\beta}, \hat{V}(\hat{\beta}))$ (King, Tomz, and Wittenberg 2000). Compare to the delta-method interval from `avg_comparisons()`.
4. **Offsets.** Suppose you wanted to model children *per year since age 18* rather than children. Add `offset(log(pmax(age - 18, 1)))` to the Poisson and interpret the coefficients. What changed?

5. **Hurdle versus zero-inflated.** Fit `pscl::zeroinfl(childs ~ educ + female + black + exper + evermar, dist = "negbin")` and add it to the AIC/BIC table. The two are not nested: which does each criterion prefer, and by how much? Then say which story fits the outcome: is a childless respondent someone who has not crossed a hurdle, or a different kind of person who could never have a positive count?
6. **A block test in the multinomial.** Drop `educ` from the region model and test it with an LR test (how many degrees of freedom?) and with BIC. Do the two agree? Explain in one sentence why they might not at $n = 3509$.
7. **Random slopes.** Extend the random-intercept model to `MathAch ~ SES + Sector + (1 + SES | School)`, so that the SES slope varies across schools. Report the variance of the slope and its correlation with the intercept, test the extension against the random-intercept model with an LR test, and say in one sentence what a school with a steep SES slope looks like.

8 Session information

```
sessionInfo()
#> R version 4.5.2 (2025-10-31)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS Tahoe 26.6.2
#>
#> Matrix products: default
#> BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
#>
#> locale:
#> [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
#>
#> time zone: America/New_York
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats      graphics  grDevices  utils      datasets  methods   base
#>
#> other attached packages:
#> [1] pscl_1.5.9          nnet_7.3-20         AER_1.2-17
#> [4] survival_3.8-3      car_3.1-5           carData_3.0-6
#> [7] MASS_7.3-65         lmtest_0.9-40       zoo_1.8-15
#> [10] sandwich_3.1-1      marginaleffects_0.32.0 ggplot2_4.0.2
#> [13] dplyr_1.2.0         lme4_2.0-1          Matrix_1.7-4
#>
```

```

#> loaded via a namespace (and not attached):
#> [1] generics_0.1.4      lattice_0.22-7      digest_0.6.39
#> [4] magrittr_2.0.4      evaluate_1.0.5      grid_4.5.2
#> [7] RColorBrewer_1.1-3  fastmap_1.2.0       jsonlite_2.0.0
#> [10] backports_1.5.0     Formula_1.2-5       tinytex_0.58
#> [13] scales_1.4.0        abind_1.4-8         reformulas_0.4.4
#> [16] Rdpack_2.6.6        cli_3.6.5           rlang_1.1.7
#> [19] rbibutils_2.4.1     splines_4.5.2       withr_3.0.2
#> [22] yaml_2.3.12         otel_0.2.0          tools_4.5.2
#> [25] checkmate_2.3.4     nloptr_2.2.1        minqa_1.2.8
#> [28] boot_1.3-32         vctrs_0.7.1         R6_2.6.1
#> [31] lifecycle_1.0.5     insight_1.4.6       pkgconfig_2.0.3
#> [34] pillar_1.11.1       gtable_0.3.6        glue_1.8.0
#> [37] data.table_1.18.2.1 Rcpp_1.1.1          xfun_0.56
#> [40] tibble_3.3.1        tidyselect_1.2.1    knitr_1.51
#> [43] farver_2.1.2        htmltools_0.5.9     nlme_3.1-168
#> [46] labeling_0.4.3      rmarkdown_2.30      compiler_4.5.2
#> [49] S7_0.2.1

```